程式語言

Programming Languages — Class 6

政大資碩 115-1　陳家宏

今天三堂課

時段	內容
第 1 堂	§3.3 PROC 的規格（pp.74–78）
第 2 堂	§3.3.2 程序的兩種表示法與實作（pp.79–80）
第 3 堂	兩個隨堂演練：Ex 3.20、Ex 3.19；HW3 發布

LET 能自己定義新運算嗎？

上週的 Ex 3.6：要讓語言有 `minus` ，得改 grammar、加 variant、加 `cases` 分支。

p.74：到目前為止，語言只有一開始放進去的運算；
要讓這個語言有用，必須能建立新的程序。這個新語言叫 PROC。

```
let minus = proc (n) -(0,n)
in (minus 5)
```

為什麼要在意（我的補充）：有了 `proc` ， `minus` 可以寫在程式裡，直譯器不必改。
這段程式第 2 堂會實際跑。

PROC 的第一個程式（p.75）

```
let f = proc (x) -(x,11)
in (f (f 77))
```

p.75：這個程式造出一個「減 11」的程序，命名為 `f`，再對 77 套用兩次，得到 55。

p.75 的第二個程式

```
(proc (f) (f (f 77))  proc (x) -(x,11))
```

p.75：第一個 proc 收一個引數，把它對 77 套用兩次；接著把這個程序套用到「減 11」的程序上，結果一樣是 55。

我的解讀	「減 11」做成 proc	「對 77 套用兩次」做成 proc
第一個程式	有	沒有
第二個程式	有	有

收程序當引數的程序，一般稱為 higher-order procedure。

```
> (run "(proc (f) (f (f 77)) proc (x) -(x,11))")
#(struct:num-val 55)
> (run "(proc (f) (f (f 77)) proc (x) -(x,1))")
#(struct:num-val 75)
```

PROC 的規格，四個角度各改了什麼？

class 5 的整理：§3.2 依序用四個小節定義 LET 的規格。

角度	LET（§3.2）	PROC（§3.3）
syntax	Figure 3.2（3.2.1, p.60）	加兩條 production：`proc-exp`、`call-exp`（p.75）
values	Int + Bool（3.2.2, p.61）	加上 Proc；Proc 是 ADT（p.75）
environments	`empty-env`、`extend-env`、`apply-env`（3.2.3, p.61）	沒有新增操作；變數可以綁到程序（由 p.75、p.79 推出）
behavior	3.2.4–3.2.7（pp.62–67）	三條等式（p.76）

§3.3 沒有分小節，pp.75–76 依序講 values → syntax → behavior。

ExpVal 與 DenVal 多了什麼？

p.75：依照 Scheme 的設計，程序也是 expressed value。

```
ExpVal = Int + Bool + Proc
DenVal = Int + Bool + Proc
```

	LET（p.61）	PROC（p.75）
ExpVal	Int + Bool	Int + Bool + Proc
DenVal	Int + Bool	Int + Bool + Proc

p.75：Proc 是表示程序的值的集合，課本把它當成一個 abstract data type。

介面見下一張。

Proc 的介面（p.75）

型別	constructors	observer	出處
Env	`empty-env` 、 `extend-env`	`apply-env`	p.36
Proc	`procedure`	`apply-procedure`	p.75

p.75：`procedure` 說明怎麼建立程序值，`apply-procedure` 說明怎麼套用程序值。

兩條新 production（p.75）

```
Expression ::= proc (Identifier) Expression
               proc-exp (var body)

Expression ::= (Expression Expression)
               call-exp (rator rand)
```

寫法	意思
`proc (x) -(x,11)`	建立一個程序
`(f 77)`	呼叫一個程序

呼叫的語法沒有關鍵字，靠一對括號辨認。

formal parameter、operand、argument 各指什麼？

p.75 的用語：

用語	指的是	在 `let f = proc (x) -(x,11) in (f -(80,3))` 裡
bound variable / formal parameter	`proc-exp` 的 `var`	`x`
operator	`call-exp` 的 `rator`	`f`
operand / actual parameter	`call-exp` 的 `rand`	`-(80,3)`
argument	actual parameter 的**值**	77

operand 是一段程式，argument 是一個值。

同一段 `proc`，求值兩次

p.75：下一步要決定，一個表示程序的值裡必須包含哪些資訊。

```
let x = 200
in let f = proc (z) -(z,x)
   in let x = 100
      in let g = proc (z) -(z,x)
         in -((f 1), (g 1))
```

p.75 用一句話描述 lexical scope rule：程序被套用時，body 在「formal parameter 綁到引數」的環境裡求值；
程序裡的自由變數也遵守 lexical binding rule。

停下來想：`f` 和 `g` 的程式碼一字不差。`(f 1)` 和 `(g 1)` 各是多少？

`procedure` 需要幾個參數？

p.76：`proc (z) -(z,x)` 求值了兩次；第一次 x 是 200，第二次 x 是 100。
兩個程序來自相同的 expression，行為卻必須不同。

所以 proc expression 的值必須依賴求值時的環境。
p.76 的結論：constructor `procedure` 收三個參數：bound variable、body、環境。

```
(value-of (proc-exp var body) ρ)
  = (proc-val (procedure var body ρ))                    ; p.76
```

p.76：`proc-val` 和 `num-val`、`bool-val` 一樣，是 ExpVal 的 constructor。

call-exp 的等式（p.76）

```
(value-of (call-exp rator rand) ρ)
  = (let ((proc (expval->proc (value-of rator ρ)))
          (arg  (value-of rand ρ)))
      (apply-procedure proc arg))
```

p.76：`expval->proc` 和 `expval->num` 一樣，先檢查 `(value-of rator ρ)` 是不是由 `proc-val` 建出來的，是的話取出裡面的程序。

步驟	結果的型別
`(value-of rator ρ)`	ExpVal
`(expval->proc ...)`	Proc
`(value-of rand ρ)`	ExpVal
`(apply-procedure proc arg)`	ExpVal

apply-procedure 的等式（p.76）

```
(apply-procedure (procedure var body ρ) val)
  = (value-of body [var=val]ρ)
```

p.76 的兩點：

1. body 在「formal parameter 綁到引數」的環境裡求值
2. 其他變數的值，要和建立程序時相同

等式右邊的 ρ 來自 `procedure` 的第三個欄位，也就是建立程序時的環境。

三條等式並排

expression	等式右邊	頁碼
`(value-of (proc-exp var body) ρ)`	`(proc-val (procedure var body ρ))`	p.76
`(value-of (call-exp rator rand) ρ)`	`(apply-procedure (expval->proc (value-of rator ρ)) (value-of rand ρ))`	p.76
`(apply-procedure (procedure var body ρ) val)`	`(value-of body [var=val]ρ)`	p.76

我的解讀：call-exp 呼叫 `apply-procedure`，`apply-procedure` 又呼叫 `value-of`，兩者交互遞迴。LET 的 `value-of` 只對子運算式遞迴；PROC 的 `apply-procedure` 求值的 body 不是子運算式。

§3.3.1 用等式算（上）

p.77：用規格計算，還沒用到實作。ρ 是任意環境。（節錄）

```
(value-of <<let x = 200 in let f = proc (z) -(z,x) in ... >> ρ)

= (value-of <<let f = proc (z) -(z,x) in let x = 100 in ... >>
    [x=⌈200⌉]ρ)

= (value-of <<let x = 100 in let g = proc (z) -(z,x) in ... >>
    [f=(proc-val (procedure z <<-(z,x)>> [x=⌈200⌉]ρ))]
    [x=⌈200⌉]ρ)

= (value-of <<let g = proc (z) -(z,x) in -((f 1), (g 1))>>
    [x=⌈100⌉]
    [f=(proc-val (procedure z <<-(z,x)>> [x=⌈200⌉]ρ))]
    [x=⌈200⌉]ρ)
```

§3.3.1 用等式算（下）

p.78（節錄，省略 `(value-of <<(f 1)>> ...)` 那一步）：

```
= ⌈(- (apply-procedure
        (procedure z <<-(z,x)>> [x=⌈200⌉]ρ)
        ⌈1⌉)
      (apply-procedure
        (procedure z <<-(z,x)>> [x=⌈100⌉][f=...][x=⌈200⌉]ρ)
        ⌈1⌉))⌉

= ⌈(- (value-of <<-(z,x)>> [z=⌈1⌉][x=⌈200⌉]ρ)
      (value-of <<-(z,x)>> [z=⌈1⌉][x=⌈100⌉][f=...][x=⌈200⌉]ρ))⌉

= ⌈(- -199 -99)⌉
= ⌈-100⌉
```

環境裡有兩個 x，`f` 會看到哪一個？

```
;; 呼叫 (f 1) 時，當下的環境 (p.78)
[g=...][x=⌈100⌉][f=...][x=⌈200⌉]ρ

;; f 綁到的值 (p.77)
(proc-val (procedure z <<-(z,x)>> [x=⌈200⌉]ρ))
```

`f` 建立時，`[x=⌈100⌉]` 這一層還不存在。`apply-procedure` 用的是 `procedure` 第三個欄位存下的環境 (p.76)，呼叫當下的環境沒有用到。

	`(f 1)`	`(g 1)`
`procedure` 的第三個欄位	`[x=⌈200⌉]ρ`	`[x=⌈100⌉][f=...][x=⌈200⌉]ρ`
body 求值的環境 (p.78)	`[z=⌈1⌉][x=⌈200⌉]ρ`	`[z=⌈1⌉][x=⌈100⌉][f=...][x=⌈200⌉]ρ`
`apply-env` 先找到的 x	200	100（最內層）
結果	−199	−99

第 1 堂回顧

1. PROC 把 Proc 加進 ExpVal 與 DenVal（p.75）
2. 兩條新 production：`proc-exp`、`call-exp`；operand 是程式，argument 是值（p.75）
3. Proc 是 ADT：constructor `procedure`、observer `apply-procedure`（p.75）
4. 程式碼相同的 `proc` 可以算出不同的程序，所以 `procedure` 要收環境（p.76）
5. `apply-procedure`：body 在 `[var=val]ρ` 裡求值，ρ 是建立程序時的環境（p.76）

第 2 堂

§3.3.2 程序的兩種表示法（pp.79–80）

§3.3.2 要做什麼？

第 1 堂只有規格。p.77 特別說明：§3.3.1 的計算用的是規格，還沒有實作。

§3.3.2 要給出 Proc 的實作，課本給了兩種：

	表示法	頁碼
1	procedural representation	p.79
2	data structure representation	p.79

p.80：兩種實作都滿足 procedure 介面的規格。

Proc 有幾個 observer？

class 3 看過的 §2.2.3（pp.40–41）：

- p.40：environment 的介面恰好只有一個 observer `apply-env`，所以可以用 Scheme 的程序表示
- p.41：只有一個 observer 的情況沒那麼少見；p.41 給了一個 recipe

型別	observer 個數
Env（第二章）	1：`apply-env`
Proc	1：`apply-procedure`

p.79 開頭直接引用 §2.2.3 的 recipe，用 procedural representation 表示程序。

procedural representation（p.79）

```
;; proc? : SchemeVal -> Bool
(define proc?
  (lambda (val)
    (procedure? val)))

;; procedure : Var × Exp × Env -> Proc
(define procedure
  (lambda (var body env)
    (lambda (val)
      (value-of body (extend-env var val env)))))

;; apply-procedure : Proc × ExpVal -> ExpVal
(define apply-procedure
  (lambda (proc1 val)
    (proc1 val)))
```

`proc?` 哪裡不準？

p.79：這裡定義的 `proc?` 不太準確，因為不是每個 Scheme 程序都是這個語言裡可能出現的程序；
它只是為了定義 `expval` 而存在。

停下來想：在兩個版本的 `data-structures.scm` 裡，各會得到什麼？

```
(proc? car)
(proc-val car)
```

data structure representation（p.79）

```
;; proc? : SchemeVal -> Bool
;; procedure : Var × Exp × Env -> Proc
(define-datatype proc proc?
  (procedure
    (var identifier?)
    (body expression?)
    (saved-env environment?)))

;; apply-procedure : Proc × ExpVal -> ExpVal
(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body (extend-env var val saved-env))))))
```

p.79：這是 §2.2.2 那種資料結構表示法。

這裡的 `define-datatype` 跟 abstract syntax 有何不同？

datatype	表示 defined language 的什麼	constructor 由誰、何時呼叫	使用端怎麼拆開
`lc-exp`	程式（AST）	`parse-expression`，parse 時	直接 `cases lc-exp`
`expression`	程式（AST）	SLLGEN 的 parser，parse 時	`value-of` 直接 `cases expression`
`expval`	值：ExpVal = Int + Bool + Proc	`value-of`，執行時	只透過 extractor
`proc`（ds-rep）	程序值：Proc，ExpVal 的一部分	`value-of`（proc-exp），執行時	只透過 `apply-procedure`

差異一：前兩列在執行前建立，描述 abstract syntax tree；後兩列在執行時建立，描述算出來的值。

差異二（我的解讀）：abstract syntax 的 variant 就是介面，`value-of` 直接做 `cases`。`proc` 的介面是 `procedure`、`apply-procedure`（p.75），`define-datatype` 只是兩種實作之一（p.79）。

closure 這個名字從哪來？

p.80：這些資料結構（ds-rep 的 `proc`）常被稱為 **closures**，因為它們自給自足：套用這個程序需要的東西，都在裡面。

p.80：我們有時說，這個程序 **closed over**（或 closed in）它被建立時的環境。

欄位（ds-rep）	套用時用來做什麼
`var`	要綁定引數的變數名稱
`body`	要求值的程式
`saved-env`	body 裡其他變數的值從這裡查

兩種表示法對照

這張表是我的整理。

	procedural（proc-rep）	data structure（ds-rep）
`procedure` 回傳	一個 Racket 的 `lambda`	一個 `procedure` variant
var、body、env 存在哪	lambda 的自由變數	variant 的三個欄位
`apply-procedure` 做什麼	呼叫那個 lambda	`cases` 拆開後求值 body
`(proc? car)`	`#t`	`#f`
REPL 印出 `proc (x) -(x,11)` 的值	`#(struct:proc-val #<procedure:...>)`	看得到 `var`、`body`、`saved-env`

最後兩列可以在課程 repo 的 `proc-lang/proc-rep` 與 `proc-lang/ds-rep` 自己跑。

data structure representation 讓 PROC 少借了什麼？

p.41：如果實作語言不支援 higher-order procedures，可以再多做一步，用 data structure representation 實作同一個介面；這個過程叫 **defunctionalization**。

我的整理：PROC 有哪些行為，是從 Racket 借來的？

PROC 的行為	proc-rep 由誰決定	ds-rep 由誰決定
程序值存了什麼	Racket 的 `lambda`	`procedure` 的三個欄位（p.79）
整數的範圍	Racket 的數字	同左
呼叫的控制流程	`value-of` 遞迴，靠 Racket 的呼叫堆疊	同左
子運算式的求值順序	`value-of` 裡 `let` 的順序	同左

ds-rep 只改變了第一列，其餘三列仍由 Racket 決定。

expval 加上 proc-val

p.80：兩種實作都要在 `expval` 加一個 variant。

```
(define-datatype expval expval?
  (num-val  (num number?))
  (bool-val (bool boolean?))
  (proc-val (proc proc?)))
```

`expval->proc` 在 p.76 只有文字描述；程式碼在課程 repo 的 `data-structures.scm`：

```
;; expval->proc : ExpVal -> Proc
(define expval->proc
  (lambda (v)
    (cases expval v
      (proc-val (proc) proc)
      (else (expval-extractor-error 'proc v)))))
```

value-of 的兩個新分支

等式（p.76）	`cases` 分支（p.80）
`(value-of (proc-exp var body) ρ)` `= (proc-val (procedure var body ρ))`	`(proc-exp (var body)` `  (proc-val (procedure var body env)))`

`(value-of (call-exp rator rand) ρ)`
`= (let ((proc (expval->proc (value-of rator ρ)))`
`  (arg (value-of rand ρ)))`
`  (apply-procedure proc arg))`
	`(call-exp (rator rand)`
	`  (let ((proc (expval->proc (value-of rator env)))`
	`  (arg (value-of rand env)))`
	`  (apply-procedure proc arg)))`

和上週一樣：等式右邊寫成分支的 body，ρ 寫成 `env`。

expval、proc、environment 繞成一圈

這張是我的解讀，依據是課程 repo `ds-rep/data-structures.scm` 的三個定義。

```
(define-datatype expval expval?
  ...
  (proc-val (proc proc?)))              ; expval 裡有 proc

(define-datatype proc proc?
  (procedure (var symbol?)
             (body expression?)
             (env environment?)))       ; proc 裡有 environment

(define environment?                    ; environment 裡有 expval
  (lambda (x)
    (or (empty-env-record? x)
        (and (pair? x)
             (symbol? (car (car x)))
             (expval? (cadr (car x)))
             (environment? (cdr x))))))
```

停下來想：class 4 的 s-list 與 s-exp 互相引用。這裡有幾個型別互相引用？LET 有這個環嗎？

現場跑：p.75 的兩個程式

```
> (run "let f = proc (x) -(x,11) in (f (f 77))")
#(struct:num-val 55)

> (run "(proc (f) (f (f 77)) proc (x) -(x,11))")
#(struct:num-val 55)

> (run "let minus = proc (n) -(0,n) in (minus 5)")
#(struct:num-val -5)
```

`proc-rep` 與 `ds-rep` 各跑一次，結果相同。

`(3 4)` 的錯誤是誰抓到的？

```
> (scan&parse "(3 4)")
#(struct:a-program #(struct:call-exp #(struct:const-exp 3) #(struct:const-exp 4)))

> (run "(3 4)")
expval-extractors: Looking for a proc, found #(struct:num-val 3)
```

停下來想：

1. parser 為什麼接受這個程式？
2. 錯誤是在 call-exp 等式的哪一步發生的？

每次擴充都要先寫規格

p.80 的 Reminder：每次擴充語言，都要寫下規格，並指回 p.70 的註記。

p.70：本書說「extend the language by adding……」，意思是兩件事：

1. 在語言的規格裡加上新的規則或等式
2. 新增或修改直譯器，實作這個功能

為什麼要在意（我的補充）：等式先寫出來，就能先手算答案，再拿直譯器的輸出對照。HW3 的每一題都照這兩件事評分。

第 2 堂回顧

1. Proc 只有一個 observer，p.79 照 §2.2.3 的 recipe 用 procedural representation 表示
2. 第二種表示法是 `define-datatype proc`，p.80 稱為 closure
3. 兩種表示法都滿足同一組規格（p.80）
4. `expval` 多一個 `proc-val`，`value-of` 多兩個分支（p.80）
5. `(3 4)` 能通過 parse；執行時，`expval->proc` 發現 `3` 不是程序才報錯（p.76）

第 3 堂

兩個隨堂演練 + HW3

Reynolds 的主張（補充教材）

（課外補充。出處：Reynolds 1972，重刊於 HOSC 11(4), 1998，pp.363–397；EOPL 只在 p.375 用一句話提到這篇論文。）

Reynolds 指出，metacircular interpreter 用 defining language 的 higher-order function 來實作 defined language 的 higher-order function；讀者如果原本就不懂這個特性，從這種定義裡很難得到解釋。

另外，直接風格的 interpreter（不限 metacircular）通常會讓 defined language 繼承 defining language 的呼叫方式：引數在呼叫前求值（call by value），或用到時才求值（call by name）。

這是兩個獨立的問題。論文接著示範：用 defunctionalization 消除前者，用 CPS 消除後者，就能得到一個 first-order、與呼叫方式無關的定義。

演練一：Exercise 3.20 [★]（p.80）

PROC 的程序只收一個參數。

p.80：讓程序回傳另一個程序，就能得到多參數的效果，例如：

```
let f = proc (x) proc (y) ...
in ((f 3) 4)
```

這個技巧叫 **Currying**，這樣的程序稱為 Curried。

寫一個 Curried 程序，接受兩個參數，回傳它們的和。

提示（p.80）：`x + y` 可以寫成 `-(x,-(0,y))`。

演練一：用 `run` 對答案

```
> (run "let plus = proc (x) proc (y) -(x,-(0,y))
        in ((plus 3) 4)")
#(struct:num-val 7)
```

`(plus 3)` 的值是什麼？

```
;; ds-rep
> (run "let plus = proc (x) proc (y) -(x,-(0,y))
        in (plus 3)")
```

在 REPL 印出來，看 `saved-env` 裡有沒有 `x`。

演練二：Exercise 3.19 [★]（p.80）

p.80：在許多語言裡，程序必須在建立的同時命名。

把 `proc` expression 換成 `letproc` expression，讓 PROC 有這個性質。

在紙上寫出：

1. 要加的 production，以及 variant 名稱與欄位
2. `value-of` 對這個 variant 的等式
3. `value-of` 裡對應的 `cases` 分支

```
letproc f (x) = -(x,11)
in (f (f 77))                          ; 這是我設計的語法，應得 55
```

分組討論

1. 分成 3 組，自由討論演練二。
2. 10 分鐘後，抽一組上台發表。

HW3：四個題號

題號	內容	難度	頁碼
3.16	一次宣告多個變數的 `let`	★★	p.74
3.17	`let*`	★★	p.74
3.23	`makemult`，再寫出 factorial	★★	p.81
3.28	dynamic binding，兩種表示法各做一次	★★	p.82

繳交期限：week 8 上課前，上傳 Moodle。

每題練到什麼

題號	如果你答得出來，代表你會了
3.16	每個右手邊都在**目前的**環境求值（p.74），p.74 的例子應得 1
3.17	同一個例子換成 `let*`，p.74 的答案是 2；差別只在右手邊在哪個環境求值
3.23	沒有遞迴的語法，也能寫出重複呼叫；p.81 提示用 Currying（演練一）寫 `times`
3.28	`apply-procedure` 改用呼叫處的環境，第 1 堂的三條等式要改哪一條

3.16 和 3.28 怎麼接起來？

p.82，Exercise 3.28 的例子：

```
let a = 3
in let p = proc (x) -(x,a)
       a = 5
   in -(a,(p 2))
```

第二個 `let` 一次宣告了 `p` 和 `a` 兩個變數，這是 3.16 的語法。

所以 3.28 的直譯器要建在 3.16 之上，才跑得動這個例子。

p.82：在 dynamic binding 下，程序 body 裡的 `a` 綁到 5；在今天的 PROC 裡，它綁到 3。

硬性要求

一、每個函式都要有 contract 與 usage 註解。（同 HW1、HW2）

二、每個輔助函式要有自己的獨立規格。（同 HW1、HW2）

三、自己補測試。 每題至少兩個課本沒給的例子，包含邊界情況。（同 HW1、HW2）

四、每一題先寫規格，再寫程式。（p.70、p.80）
在程式碼上方寫出新的 production 與 `value-of` 等式：

```
;; Expression ::= let* {Identifier = Expression}* in Expression
;;                let*-exp (vars exps body)
;; (value-of (let*-exp ...) ρ) = ...
```

五、3.28 兩種表示法都要跑 p.82 的例子，附上結果。

下週預告：§3.4 LETREC（p.82 起）

用今天的 PROC 跑跑看：

```
> (run "let double = proc (x) if zero?(x)
                             then 0
                             else -((double -(x,1)), -2)
        in (double 6)")
apply-env: No binding for double
```

p.82 用新的語法寫同一個程序：

```
letrec double(x)
         = if zero?(x) then 0 else -((double -(x,1)), -2)
in (double 6)
```

下週要回答：`proc` 的 body 裡出現 `double` 時，它的環境裡為什麼沒有 `double`？

本日結論

1. 程式碼相同的 `proc` 可以算出不同的程序，所以 `procedure` 收三個參數：var、body、環境（p.76）
2. `apply-procedure`：body 在 `[var=val]ρ` 裡求值，ρ 是建立程序時存下來的環境（p.76）
3. Proc 只有一個 observer，可以用 procedural 或 data structure 表示；後者稱為 closure（pp.79–80）
4. ds-rep 讓程序值不再借用 Racket 的 closure（p.41、p.79）；其餘行為仍由 Racket 決定（我的整理）
5. 擴充語言 = 先寫規則，再改直譯器（p.70）

問題時間

1. Email: laurence@replware.dev
2. Office Hour: 週四下午 1:00~2:00，請先跟我約
3. 助教：邵振皓 (負責改作業、監考)
4. 助教信箱：114971020@nccu.edu.tw
5. 作業一律上傳 Moodle